Objects I — Compound Data

Lecture 28

CS 1001 Kirk Duran

Today

- Explain why related values sometimes need to represent one conceptual entity.
- Define **class**, **instance**, **attribute**, and **object state**.
- Define a simple class and initialize instance attributes with `__init__`.
- Construct multiple independent instances from the same class definition.
- Read and update attributes using dot notation.
- Extend our existing name–reference model from lists to user-defined objects.
- Compare an object representation with a dictionary representation of the same data.

Key idea

An object lets several related values collectively represent one thing; a class describes the structure shared by objects of that kind.

The Boundary for This Lecture

Today

- compound data
- classes and instances
- construction and initialization
- instance attributes
- object state
- references to objects
- object vs. dictionary representation

Next lecture

- general methods
- deeper treatment of `self`
- mutation through methods
- object aliasing
- identity and equality
- lists of objects
- objects containing collections

Part 1: Why Compound Data?

One Entity Can Require Several Related Values

Suppose a program needs to represent two cars.

Separate variables

```
car1_make = "Honda"
car1_model = "Civic"
car1_year = 2020

car2_make = "Toyota"
car2_model = "Corolla"
car2_year = 2022
```

- The program contains the necessary values.
- Their relationship exists only in the variable names.
- Adding another car requires another group of coordinated names.

A List Groups the Values but Hides Their Roles

List representation

```
car1 = ["Honda", "Civic", 2020]
```

- One value now contains the three related components.
- But their meaning depends on remembering a positional convention.
- `car1[0]` means make only because the programmer decided that index 0 means make.



Index	Value	Meaning
0	Honda	make
1	Civic	model
2	2020	year

A Dictionary Gives the Components Meaningful Names

Dictionary representation

```
car1 = {"make":  "Honda", "model":  "Civic", "year":  2020}
```

Access

```
car1["make"]
car1["model"]
car1["year"]
```

- The keys document each component's role.
- The values are still grouped into one structure.
- This is often an excellent representation for associative data.

Definition: Compound Data

Definition

Compound data represents one conceptual value using multiple related component values.

- A date might contain a year, month, and day.
- A student might contain a name, major, and GPA.
- A point might contain an x coordinate and a y coordinate.
- A car might contain a make, model, and year.

New question

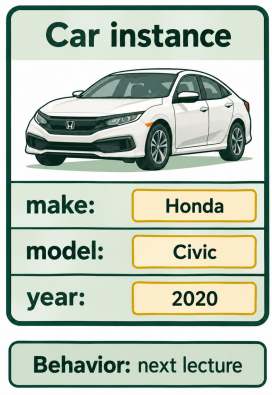
How can we define a reusable structure that says what components every *Car* should contain?

Objects Combine Related State Under One Identity

Definition

An **object** is a value with its own identity and state. Its state can contain multiple related attributes.

- **Attributes** store information associated with the object.
- Objects can also have behavior through methods.
- Today we focus on the **state** side of this model.



Part 2: Classes and Instances

A Class Describes a Kind of Object

Definition

A **class** is a program definition that describes the structure and behavior shared by objects of a particular kind.

- The class is a reusable definition.
- A class does not describe one particular Honda or Toyota.
- It describes what information each `Car` instance should maintain.

Key idea

Think **class = definition of a kind**; **instance = one particular value of that kind**.

First Class Definition

Car class

```
class Car:
   def __init__(self, make:  str, model:  str, year:  int):
      self.make = make
      self.model = model
      self.year = year
```

- `class Car:` begins the definition of a new class.
- The indented code belongs to that class definition.
- `__init__` establishes the initial state of a new instance.
- Type hints document intended types; Python does not enforce them automatically.

An Instance Is One Particular Object

Definition

An **instance** is an individual object created according to a class definition.

Construct two instances

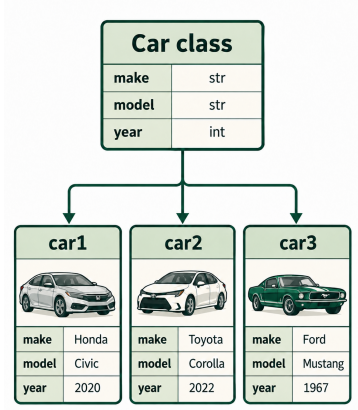
```
car1 = Car("Honda", "Civic", 2020)
car2 = Car("Toyota", "Corolla", 2022)
```

- Both objects are instances of `Car`.
- Each instance has its own attribute state.
- The variable names `car1` and `car2` refer to different instances.

One Class, Many Possible Instances

Construction

```
car1 = Car("Honda", "Civic", 2020)
car2 = Car("Toyota", "Corolla", 2022)
car3 = Car("Ford", "Mustang", 1967)
```



Part 3: Construction and Initialization

Calling the Class Constructs an Instance

Expression

```
Car("Honda", "Civic", 2020)
```

This expression produces a new `Car` instance.

1. Python creates a new instance of the class.
2. Python invokes the class's `__init__` method for that instance.
3. The supplied arguments initialize the instance's attributes.
4. The resulting instance becomes the value of the expression.

Key idea

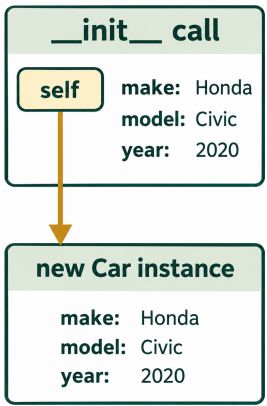
We construct an object by calling the class. `__init__` initializes the state of the newly created instance.

What Does `self` Mean Here?

- `self` is a parameter of the method.
- During initialization, it refers to the new instance being initialized.
- `self.make` means "the `make` attribute belonging to this instance."
- Different constructor calls use different instances as `self`.

For today

We only need enough `self` to understand initialization. Lecture 29 develops the method-call model in detail.



Attribute Assignment Creates the Instance State

Consider one line from `__init__`:

Statement

```
self.make = make
```

1. Evaluate the right-hand side name `make`.
2. Follow `self` to the current instance.
3. Associate the attribute name `make` with that value on the instance.

Key idea

The assignment model has not disappeared: evaluate the right-hand side, then update program state. The destination is now an attribute on an object.

Trace the First Construction

Statement

```
car1 = Car("Honda", "Civic", 2020)
```

Name during initialization	Value
`make`	`"Honda"`
`model`	`"Civic"`
`year`	2020
`self`	new `Car` instance

After the assignments, the instance contains three attributes: `make`, `model`, and `year`.

Part 4: Attributes and Object State

Dot Notation Accesses an Attribute

Given

```
car1 = Car("Honda", "Civic", 2020)
```

Expressions

`car1.make` → `"Honda"`
`car1.model` → `"Civic"`
`car1.year` → `2020`

- The expression left of the dot identifies an object.
- The name right of the dot identifies one attribute on that object.

Object State Is the Collection of Current Attribute Values

Definition

The **state** of an object is the collection of information currently stored in its attributes.

State of `car1`

`make = "Honda"`, `model = "Civic"`, `year = 2020`

- State can be inspected through attribute access.
- For mutable objects, state can change over time.
- The object's identity can remain the same while its state changes.

Attribute Assignment Changes State

Before

`car1.year` $\rightarrow$ `2020`

Statement

```
car1.year = 2021
```

After

`car1.year` $\rightarrow$ `2021`

Key idea

The name `car1` still refers to the same instance; one component of that instance's state has changed.

Mutation and Rebinding Are Still Different

Attribute mutation

```
car1.year = 2021
```

The existing instance changes state.

Name rebinding

```
car1 = Car("Ford", "Focus", 2018)
```

`car1` is bound to a different instance.

Key idea

This is the same distinction we used for lists: changing a value through a reference is different from changing which value a name refers to.

Prediction: Which Object Changes?

In class

Predict every printed value before tracing the code.

Code

```
first = Car("Honda", "Civic", 2020)
second = Car("Honda", "Civic", 2020)
first.year = 2025

print(first.year)
print(second.year)
```

- How many `Car` instances were constructed?
- Does having the same attribute values make them one instance?

Reveal: Same State Does Not Mean Same Instance

The output is:

Output

```
2025
2020
```

Two calls to `Car(...)` constructed two distinct objects, even though their initial attribute values were equal.

`first` → instance A `second` → instance B

Part 5: References and Identity

Our Earlier Reference Model Now Generalizes

Earlier in the course we used a focused vocabulary for mutable lists:

$$\text{name} \longrightarrow \text{reference} \longrightarrow \text{list}$$

We can now generalize that model:

$$\text{name} \longrightarrow \text{reference} \longrightarrow \text{object}$$

Key idea

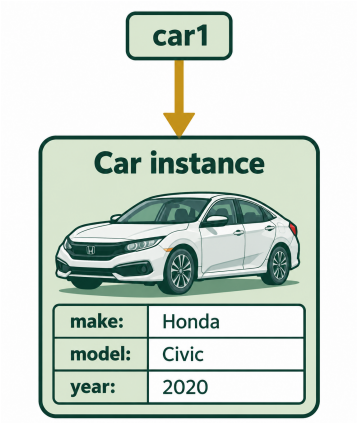
Lists were not a special exception. They are one kind of object. User-defined class instances participate in the same reference model.

A Variable Stores a Reference to the Instance

Code

```
car1 = Car("Honda", "Civic", 2020)
```

- The class call constructs a `Car` instance.
- Assignment binds the name `car1` to that resulting object.
- Access through `car1` follows that reference to the instance.



Identity Asks Whether Two References Reach the Same Object

Definition

Object identity distinguishes one particular object from every other object, even when their current state happens to look the same.

Example

```
a = Car("Honda", "Civic", 2020)
b = Car("Honda", "Civic", 2020)
a is b → False
```

Next lecture

We will use identity to reason about aliasing when two names actually refer to the same instance.

Part 6: Objects as a Data Representation

Dictionary and Object Can Represent Similar Information

Dictionary

```
car = {
  "make":  "Honda",
  "model":  "Civic",
  "year":  2020
}

car["year"]
```

Object

```
car = Car(
  "Honda",
  "Civic",
  2020
)

car.year
```

Object vs. Dictionary: What Changes?

Question	Dictionary	Class instance
field name	key	attribute
access	`data["year"]`	`car.year`
structure	convention among keys	described by class
create another	new dictionary	call class again
future behavior	separate functions	methods can belong to class

Key idea

Dictionaries remain useful. Classes become useful when the program benefits from defining a reusable kind of entity with a known structure and, eventually, associated behavior.

Part 7: Connecting to Python You Already Know

You Have Already Used Objects

Python's built-in data types also produce objects.

List

```
values = [10, 20, 30]
values.append(40)
```

String

```
text = "hello"
text.upper()
```

- We previously concentrated on the sequence behavior we needed.
- Now we have enough vocabulary to say that lists and strings are objects with methods provided by their classes.

Dot Notation Has Two Closely Related Roles

Attribute access

```
car1.year
```

Retrieve data associated with an object.

Method call

```
values.append(40)
```

Invoke behavior associated with an object.

Key idea

Today we defined our own attributes. Next lecture we define our own methods and connect method calls to particular instances.

Design Prompt: What Should a `Student` Store?

In class

We want one value to represent one student in a course. What attributes belong to that object?

Possible design:

Class

```
class Student:
   def __init__(self, name:  str, major:  str, gpa:  float):
      self.name = name
      self.major = major
      self.gpa = gpa
```

The Data Model Should Match the Concept

A good class does not simply collect every value a program might ever use.

- Attributes should describe the state of the conceptual entity.
- Temporary calculations usually do not need to become attributes.
- Two attributes should not redundantly store the same information unless the design requires it.
- Attribute names should communicate what each component means.

Example

A student's `name`, `major`, and `gpa` describe student state. A temporary variable such as `new_gpa` used during a calculation does not automatically belong in the object.

Common Misconception: Attributes Are Local Variables

Inside `__init__`:

Different roles

`make` — parameter/local name for this call
`self.make` — attribute stored on the instance

After `__init__` returns, the parameter name `make` is gone with that call frame, but `self.make` remains part of the object's state.

Key idea

Parameters help initialize the object; attributes preserve the resulting state beyond the initialization call.

Trace Checklist for Object Construction

When you encounter:

Statement

```
name = ClassName(argument1, argument2, ...)
```

trace it in this order:

1. evaluate the argument expressions;
2. construct a new instance;
3. bind initialization parameters;
4. let `self` refer to the new instance;
5. execute the attribute assignments;
6. finish the class-call expression with the initialized instance;
7. bind the result to the variable name.

Where We Stop Today

By the end of Lecture 28, you should be able to:

- explain why an object is useful for compound data;
- distinguish a class from an instance;
- read a simple class containing `__init__`;
- trace construction of a new instance;
- identify parameter names versus instance attributes;
- access and update attributes with dot notation;
- draw names as references to distinct object instances;
- compare an object representation with a dictionary representation.

Takeaways

1. Compound data represents one conceptual value using several related components.
2. A class defines a reusable kind of object; an instance is one particular object of that class.
3. Calling a class constructs a new instance and `__init__` establishes its initial state.
4. Attributes are named components of object state and are accessed with dot notation.
5. Separate constructor calls produce separate object identities, even when their state looks the same.
6. The reference model developed for lists generalizes naturally to user-defined objects.

Next: Objects II

Methods connect behavior to specific instances; aliasing and collections determine how those instances interact with the rest of a program.